Reproducing the Redis Case Study of Temeraire

TODO: Vorname Nachname
TODO:email@hhu.de
Heinrich-Heine-Universität
Düsseldorf, Nordrhein-Westfalen, Deutschland

Abstract

Temeraire extends TCMalloc with hugepage-aware placement decisions. The OSDI 2021 paper reports application-level gains and fleet-scale savings, but only a small part of that evaluation is reproducible with public code. This report therefore targets the Redis 6.0.9 case study from the paper. The seminar artifact builds two allocator variants from a historical public TCMalloc revision, runs Redis inside a Docker-based environment, and records benchmark outputs together with system metadata. The current repository already contains a smoke test that validates the benchmark harness and allocator switching. That smoke test shows higher throughput for the Temeraire build than for the legacy pageheap build, but it uses reduced parameters and does not observe huge-page backing. The result is therefore a reproducible artifact and an honest experiment design, not yet a full confirmation of the paper's performance claims.

Keywords

memory allocation, huge pages, TCMalloc, Temeraire, Redis, reproducibility

1 Introduction

The seminar task is to reproduce experiments from a systems paper and to explain both the method and the result clearly. For Temeraire, that requirement matters because the headline claim of the paper is much broader than what a public reproduction can support.

Hunter et al. present Temeraire as a hugepage-aware extension of TCMalloc that aims to improve performance beyond allocator-local efficiency [3]. The original paper studies single applications, a fleet experiment, and a production rollout. Only the Redis case study is a realistic target for a seminar artifact built from public code.

This report follows that narrow target. It first explains why huge pages matter for allocator design. It then defines the exact reproduction scope, describes the artifact and benchmark method, and discusses the current smoke-test results. The central claim is intentionally modest: the repository reproduces the Redis experiment *procedure* in a transparent way, but it does not yet reproduce the paper's production-scale evidence.

2 Background and Target Experiment

Processors translate virtual addresses through page tables and cache recent translations in translation lookaside buffers, or TLBs. If an application uses many small pages, TLB coverage can become a bottleneck. Linux Transparent Huge Pages try to reduce that pressure by backing memory with larger pages when the system can do so efficiently [8].

Huge pages are useful only when the system can keep enough memory contiguous. Fragmentation works against that goal. The allocator therefore matters: if it spreads live objects across many hugepage-sized regions, the kernel may fail to promote memory to huge pages or may need to split existing mappings later [3].

Temeraire changes the TCMalloc pageheap so that allocation decisions become aware of hugepage-sized regions. The paper's intuition is simple. Better packing can increase huge-page coverage and reduce translation overhead, even if the allocator itself does slightly more work [3].

For this seminar, the relevant paper experiment is the Redis case study. The paper evaluates Redis 6.0.9 with Temeraire and with the legacy TCMalloc pageheap. Each configuration is exercised with repeated `redis-benchmark` runs over two list operations: pushing five values with `LPUSH` and reading them back with `LRANGE` [3, 7].

This report therefore excludes the paper's fleet experiment and rollout claims. Those parts depend on Google's production environment and telemetry. The seminar artifact can only answer a smaller question: can the public Redis experiment be reconstructed in a way that is transparent, repeatable, and close to the paper's method?

3 Related Work

Temeraire is not the first allocator project to connect memory management with whole-program performance. Hoard is a classic scalable allocator for multithreaded systems. Its design addresses contention, false sharing, and memory blowup on multiprocessors [1]. That makes it useful background here, but Hoard optimizes scalability of allocation itself rather than huge-page coverage.

Another neighboring line of work studies fragmentation more directly. Mesh is a drop-in allocator for C and C++ that reduces fragmentation by "meshing" virtual spans with non-overlapping live objects [6]. Mesh and Temeraire both care about memory layout, but they optimize different outcomes. Mesh aims to recover fragmented memory without relocating objects. Temeraire instead tries to place allocations so that hugepage-sized regions remain usable.

There is also related work at the operating-system level. Navarro et al. show that transparent superpage support can substantially improve performance, while also emphasizing that promotion and fragmentation control are difficult system problems [4]. More recent work such as HawkEye revisits huge-page management inside the kernel and studies how tracking and promotion policy affect both performance and memory bloat [5]. In contrast, Temeraire changes allocator placement decisions so that the operating system has a better chance of using huge pages later.

Table 1: Smoke-test throughput from the current repository state.

Allocator	Operation	Mean RPS	Change vs. legacy
legacy	LPUSH five values	336000.00	baseline
temeraire	LPUSH five values	501999.97	+49.4%
legacy	LRANGE five values	336000.00	baseline
temeraire	LRANGE five values	503999.97	+50.0%

4 Artifact and Method

The repository is organized as a Docker-based artifact. Containerization does not make performance numbers identical across machines, but it does make the setup reproducible: source versions, build steps, benchmark commands, and raw outputs are all pinned and stored inside the repository.

Three software references define the experiment. Redis is pinned to version `6.0.9`. The allocator code comes from the public TCMalloc repository at historical commit `8e534f50707469baac732559494559db95732e12` [2]. That revision still exposes the `want_no_hpaa` mechanism, which is needed to construct a legacy baseline next to the hugepage-aware path.

The setup script builds two shared libraries from that code base: `libtcmalloc_legacy.so` and `libtcmalloc_temeraire.so`. Redis is started with one of them through `LD_PRELOAD`. Because Redis may still report `mem_allocator: libc` in its own metadata, the artifact also includes a preload check that inspects `/proc/$pid/maps`. That check is needed to verify that the intended allocator really backs the process.

The paper-shaped workflow is:

```
docker compose build
docker compose run --rm temeraire-dev bash -lc "./scripts/
     setup_env.sh"
docker compose run --rm temeraire-dev bash -lc "./scripts/
     check_allocator_preload.sh"
docker compose run --rm temeraire-dev bash -lc "./scripts/
     collect_system_info.sh"
docker compose run --rm temeraire-dev bash -lc "./scripts/
     run_redis_benchmark.sh legacy"
docker compose run --rm temeraire-dev bash -lc "./scripts/
     run_redis_benchmark.sh temeraire"
```

The full benchmark configuration follows the shape reported in the paper: 2000 trials per allocator mode, 1,000,000 requests per trial, 50 clients, and pipeline depth 16. Before and after each run, the scripts record system and process metadata such as kernel version, CPU model, Transparent Huge Page settings, Redis version, and memory summaries. Recording this context is important because Linux THP behavior depends on the surrounding system, not only on the benchmarked process [8].

5 Current Results

The repository does not yet contain a full paper-scale measurement run. It does contain one smoke test for each allocator mode. These runs were executed with reduced settings to validate the harness quickly: 1 trial, 1000 requests, 10 clients, and pipeline depth 4.

Table 1 summarizes the stored results.

At first glance, the smoke test looks favorable to Temeraire. That reading would be misleading. Each mean is based on exactly one trial, so there is no variance estimate. More importantly, both runs report `AnonHugePages: 0 kB` in their memory summaries. The test therefore validates that the harness works and that the allocator switch changes runtime behavior, but it does not yet demonstrate the huge-page effects that motivate Temeraire in the paper.

The smoke test is still useful. It shows that both allocator modes build, load, and complete the Redis workload end to end. That makes it a methodological checkpoint before running the much more expensive full experiment.

6 Discussion

The strongest property of the artifact is reproducibility of procedure. The repository fixes software versions, stores raw benchmark outputs, documents the benchmark command sequence, and collects system metadata that would otherwise be easy to forget. For a seminar project, this is important because it allows another student to inspect the exact path from source checkout to measurement.

The weakest property is fidelity to the original paper environment. The OSDI evaluation used Google's infrastructure and, for the Redis case study, specific server hardware. A local Docker deployment on commodity hardware cannot reproduce the same kernel state, memory fragmentation, NUMA effects, or performance-counter behavior. Those differences matter precisely because hugepage allocation depends on global memory conditions [8].

There is a second limit on interpretation. The public TCMalloc commit used here is a historical approximation that still supports a legacy-versus-Temeraire split. It is suitable for a seminar reproduction, but it should not be presented as proof that the internal Google artifact was rebuilt byte for byte.

For the written seminar report, the fairest interpretation is therefore this: the artifact reproduces the Redis case-study methodology with public code, while the current stored results are only smoke-test evidence. A convincing final evaluation still requires the full parameter set, repeated trials, and a discussion of variance and THP state.

7 Conclusion

This seminar artifact takes the right narrow target. Instead of claiming to reproduce Temeraire as a whole, it focuses on the Redis case study that can be rebuilt from public code. The current repository already demonstrates the full measurement pipeline and verifies that allocator switching works as intended.

The next step is straightforward: run the full Redis benchmark for both allocator modes, aggregate the repeated trials, and discuss the outcome against the paper's scope and limitations. Even if the final numbers differ from the paper, the artifact is valuable when it makes those differences explainable and reproducible.

References

[1] Emery D. Berger, Kathryn S. McKinley, Robert D. Blumofe, and Paul R. Wilson. 2000. Hoard: A Scalable Memory Allocator for Multithreaded Applications. In *Proceedings of the Ninth International Conference on Architectural Support for Programming Languages and Operating Systems*. ACM, 117–128. doi:10.1145/378993.379232

[2] Google. 2026. google/tcmalloc. https://github.com/google/tcmalloc Public repository accessed 2026-05-09.

[3] A.H. Hunter, Chris Kennelly, Paul Turner, Darryl Gove, Tipp Moseley, and Parthasarathy Ranganathan. 2021. Beyond malloc efficiency to fleet efficiency: a hugepage-aware memory allocator. In *15th USENIX Symposium on Operating Systems Design and Implementation (OSDI 21)*. USENIX Association, 257–273. https://www.usenix.org/conference/osdi21/presentation/hunter

[4] Juan Navarro, Sitaram Iyer, Peter Druschel, and Alan Cox. 2002. Practical, Transparent Operating System Support for Superpages. In *5th Symposium on Operating Systems Design and Implementation (OSDI 02)*. USENIX Association, 89–104. https://www.usenix.org/conference/osdi-02/practical-transparent-operating-system-support-superpages

[5] Ashish Panwar, Sorav Bansal, and K. Gopinath. 2019. HawkEye: Efficient Fine-grained OS Support for Huge Pages. In *Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems*. ACM, 347–360. doi:10.1145/3297858.3304064

[6] Bobby Powers, David Tench, Emery D. Berger, and Andrew McGregor. 2019. Mesh: Compacting Memory Management for C/C++ Applications. In *Proceedings of the 40th ACM SIGPLAN Conference on Programming Language Design and Implementation*. ACM, 333–346. doi:10.1145/3314221.3314582

[7] Redis Documentation. 2026. Redis benchmark. https://redis.io/docs/latest/operate/oss_and_stack/management/optimization/benchmarks/ Accessed 2026-05-09.

[8] The Linux Kernel Documentation. 2026. Transparent Hugepage Support. https://docs.kernel.org/admin-guide/mm/transhuge.html Accessed 2026-05-09.